

Brief Python

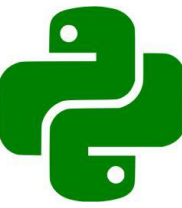
CS 30 v 2.0

First 4-week Python Course
for Beginners

Chapter 2: Conditionals

Dr. Eric Chou, IEEE Senior
Member



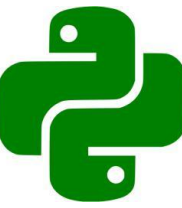


Objectives

- Flow Chart
- Boolean Value
- Character and String
- Boolean Expression
- Conditional Structures

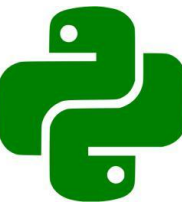
Flow Chart

LECTURE 1



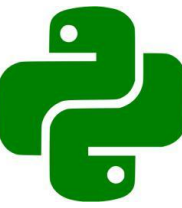
What is Flow Chart?

- Flow Chart is a graphical representation of an algorithm.
- Programmers often use it as a program-planning tool to solve a problem. It makes use of symbols which are connected among them to indicate the flow of information processing.
- The process of drawing a flowchart for an algorithm is known as “flowcharting”.



Basic Symbols used in Flowchart Designs

1. Terminals
2. Input/Output
3. Processing
4. Decision
5. Connector

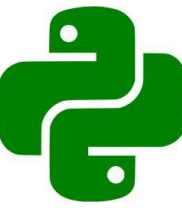


Terminal

- The oval symbol indicates Start, Stop, and Halt in a program's logic flow.
- A pause/halt is generally used in a program logic under error conditions. Terminals is the first and last symbols in a flow chart.



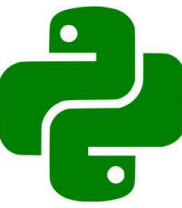
Terminal



Input/Output

- A parallelogram denotes any function of input/output type. Program instructions that take input from input devices and display output on output devices are indicated with parallelogram in a flowchart.



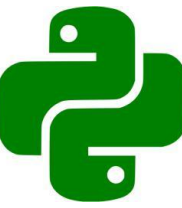


Processing

- A box represents arithmetic instructions. All arithmetic processes such as adding, subtracting, multiplication and division are indicated by action and process symbol.



Process

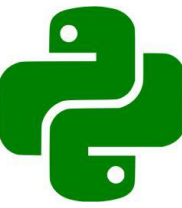


Decision

- Diamond symbol represents a decision point. Decision based operations such as yes/no question or true/false are indicated by diamond in a flowchart.



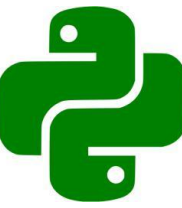
Decision



Connectors

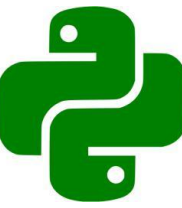
- Whenever flowchart becomes complex or it spreads over more than one page, it is useful to use connectors to avoid any confusions. It is represented by a circle.





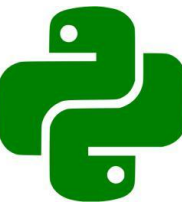
Flow Lines

- Flow lines indicate that the exact sequence in which instructions are executed. Arrows represent the direction of flow of control and relationship among different symbols of flowchart.



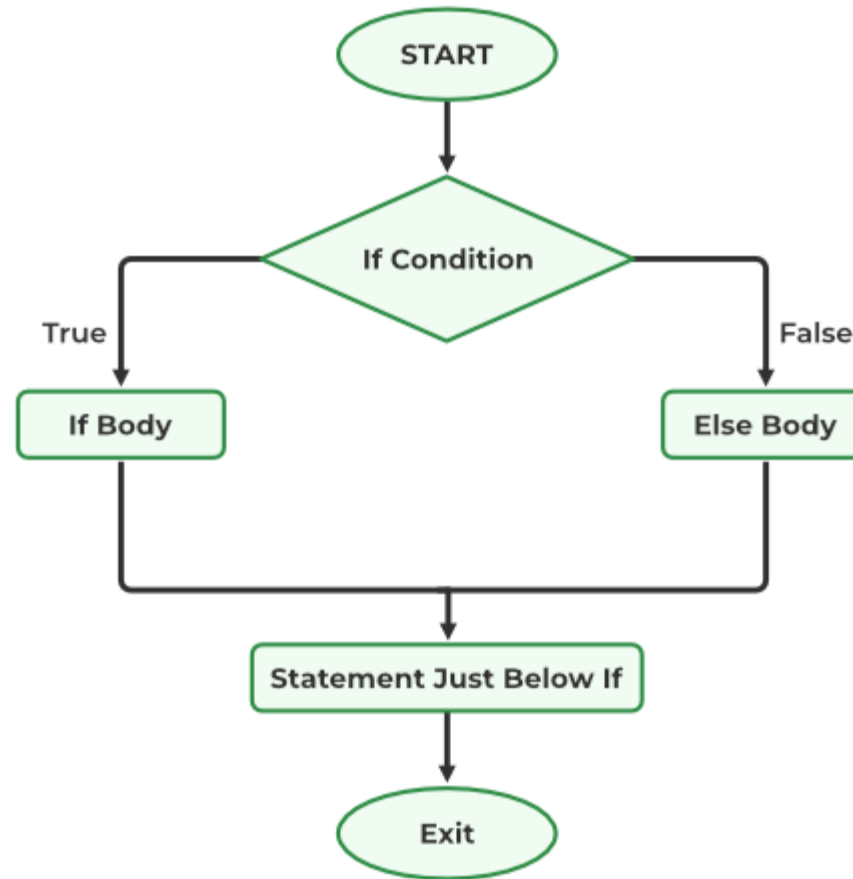
Exercise

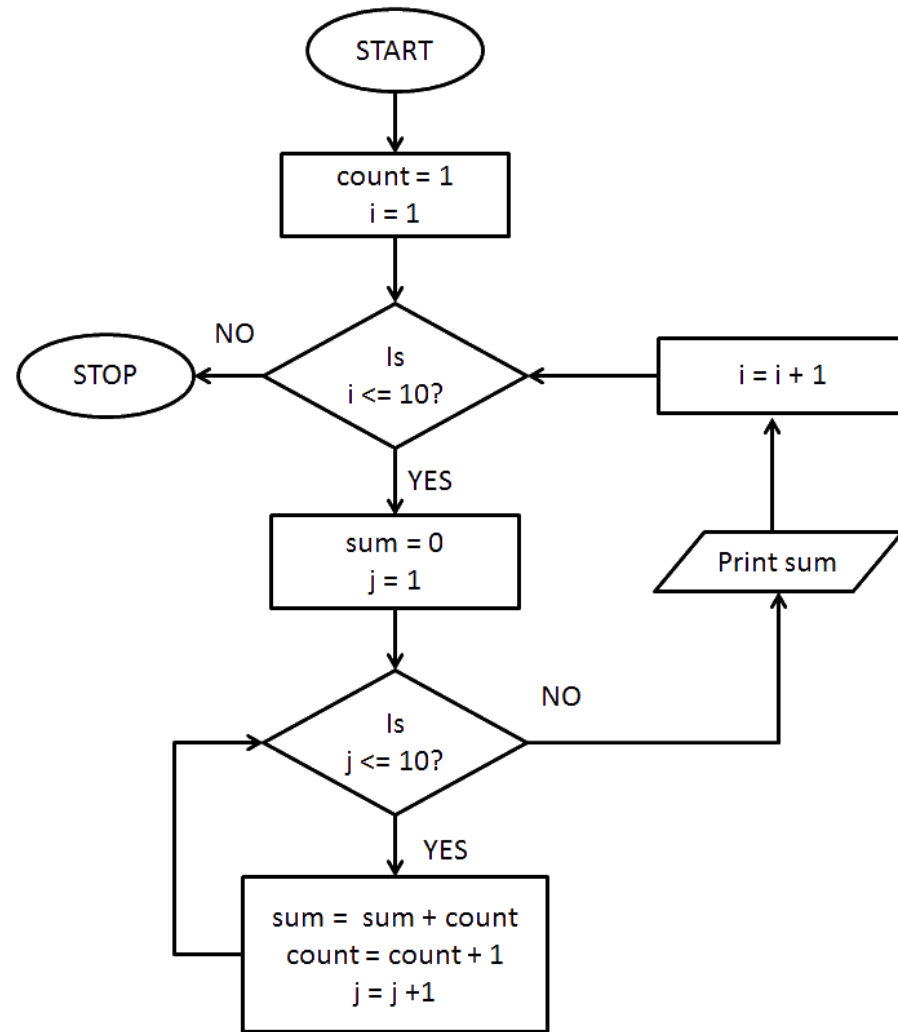
- Draw a flow chart for your daily work schedule.



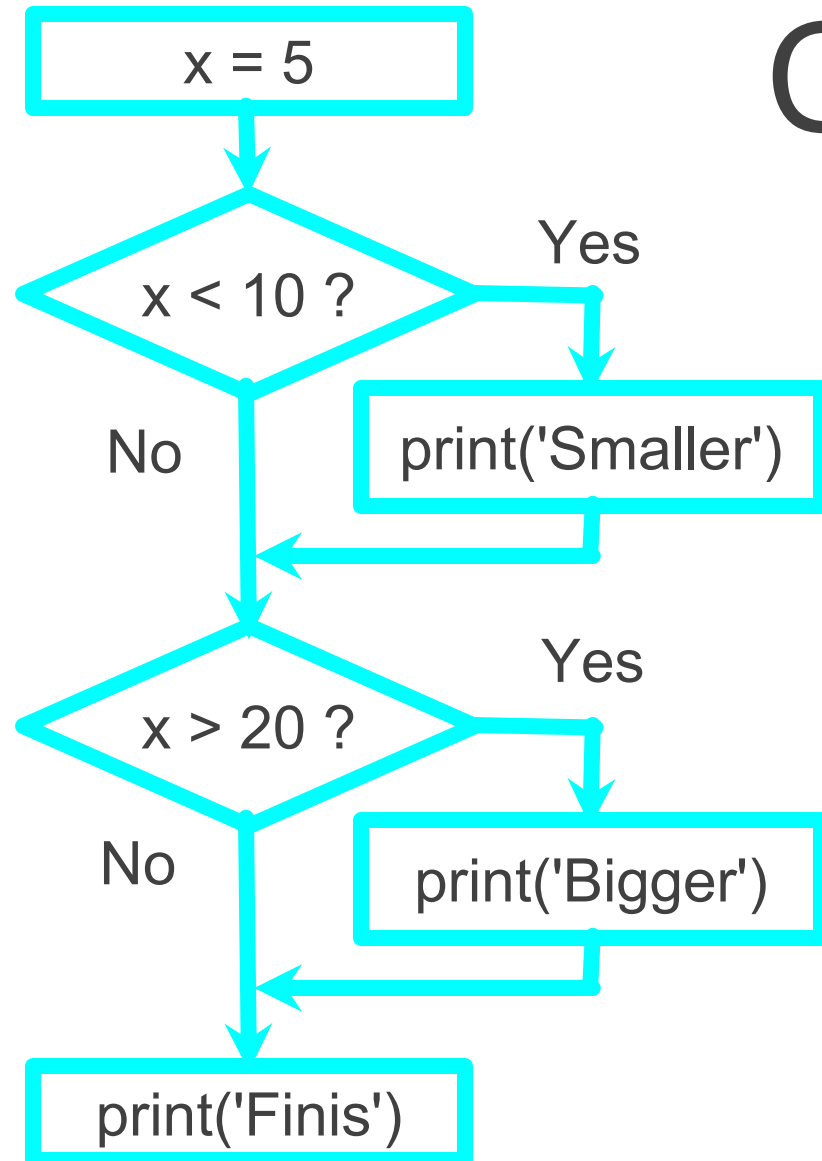
Exercise

- Draw a flow chart about what will happen if you go to the library to borrow a book. You search on a computer. If the book exists, it may be available or borrowed. Then, what will you do? Also, if the book does not exist, then, what will you do.





Conditional Steps



Program:

```
x = 5
if x < 10:
    print('Smaller')
if x > 20:
    print('Bigger')

print('Finis')
```

Output:

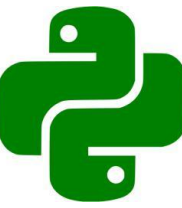
Smaller
Finis

Boolean

LECTURE 2

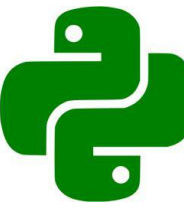


Boolean Value



Boolean Value

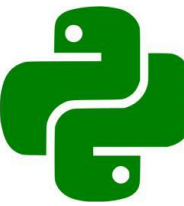
- Boolean Expressions ask a question and produce a Yes or No result which we use to control program flow.
- Boolean expressions using comparison operators evaluate to True/False or Yes/No
- Comparison operators look at variables but do not change the variables.



Comparison Operators

Python	Meaning
<	Less than
<=	Less than or Equal to
==	Equal to
>=	Greater than or Equal to
>	Greater than
!=	Not equal

Remember: “=” is used for assignment.



Comparison Operators

```
x = 5
if x == 5 :
    print('Equals 5')
if x > 4 :
    print('Greater than 4')
if x >= 5 :
    print('Greater than or Equals 5')
if x < 6 : print('Less than 6')
if x <= 5 :
    print('Less than or Equals 5')
if x != 6 :
    print('Not equal 6')
```

Equals 5

Greater than 4

Greater than or Equals 5

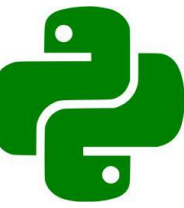
Less than 6

Less than or Equals 5

Not equal 6

Character

LECTURE 3



ASCII

American Standard Code for Information Interexchange

- Americans came up with (b-bit) ASCII representation with English only alphabets as a standard to exchange information.
- 'A' – 65 – 0x41, 'a' – 97 – 0x61

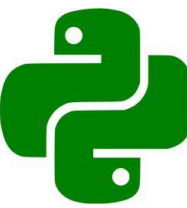
```
>> ord('A')
```

```
65
```

```
>> chr(65)
```

```
A
```

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00	NUL 000	STX 001	SOT 002	ETX 003	EOT 004	ENQ 005	ACK 006	BEL 007	BS 008	HT 009	LF 010	VT 011	FF 012	CR 013	SO 014	SI 015
10	DLE 016	DC1 017	DC2 018	DC3 019	DC4 020	NAK 021	SYN 022	ETB 023	CAN 024	EM 025	SUB 026	ESC 027	FS 028	GS 029	RS 030	US 031
20	SP 032	!	#	\$	%	&	(	)	*	+	,	-	.	/		
30	0 048	1 049	2 050	3 051	4 052	5 053	6 054	7 055	8 056	9 057	:	;	<	=	>	?
40	@ 064	A 065	B 066	C 067	D 068	E 069	F 070	G 071	H 072	I 073	J 074	K 075	L 076	M 077	N 078	O 079
50	P 080	Q 081	R 082	S 083	T 084	U 085	V 086	W 087	X 088	Y 089	Z 090	[091	] 092			
60	. 096	a 097	b 098	c 099	d 100	e 101	f 102	g 103	h 104	i 105	j 106	k 107	l 108	m 109	n 110	o 111
70	p 112	q 113	r 114	s 115	t 116	u 117	v 118	w 119	x 120	y 121	z 122	{ 123	} 124	~ 125		
80	Ж 128	Г 129	, 8218	f 402	" 8222	... 8230	† 8224	Y 8225	X 710	У 8240	Љ 352	< 8249	И 338	К 141	Г 142	И 143
90	Ж 144	' 8216	' 8217	" 8220	" 8221	' 8226	- 8211	- 8212	~ 732	™ 8482	Љ 353	' 8250	и 339	к 157	г 158	и 376
A0	° 160	± 161	l 162	i 163	г 164	ø 165	¶ 166	· 167	ž 168	№ 169	» 170	» 171	j 172	S 173	s 174	Г 175
B0	A 176	Б 177	В 178	Г 179	Д 180	Е 181	Ж 182	З 183	И 184	Й 185	К 186	Л 187	М 188	Н 189	О 190	П 191
C0	P 192	C 193	T 194	Y 195	Ф 196	X 197	Ц 198	Ч 199	Ш 200	Щ 201	Ъ 202	Ы 203	Ь 204	Э 205	Ю 206	Я 207
D0	а 208	б 209	в 210	г 211	д 212	е 213	ж 214	з 215	и 216	й 217	к 218	л 219	м 220	н 221	о 222	п 223
E0	р 224	с 225	т 226	у 227	ф 228	х 229	ц 230	ч 231	ш 232	щ 233	ъ 234	ы 235	ь 236	э 237	ю 238	я 239
F0																
	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255



Unicode

Extension of ASCII to accommodate international character sets.

UTF-8, UTF-16, UTF-32

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	
10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	
20	21	22	23	24	25	26	27	28	29	2A	2B	2C	2D	2E	2F	
30	31	32	33	34	35	36	37	38	39	3A	3B	3C	3D	3E	3F	
40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F	
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	
60	61	62	63	64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	
70	71	72	73	74	75	76	77	78	79	7A	7B	7C	7D	7E	7F	
80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F	
90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F	
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	
B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	
C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF	
D0	D1	D2	D3	D4	D5	D6	D7	D8	D9	DA	DB	DC	DD	DE	DF	
E0	E1	E2	E3	E4	E5	E6	E7	E8	E9	EA	EB	EC	ED	EE	EF	
F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	FA	FB	FC	FD	FE	FF	

Latin script

Non-Latin European scripts

African scripts

Middle Eastern and Southwest Asian scripts

South and Central Asian scripts

Southeast Asian scripts

East Asian scripts

CJK characters

Indonesian and Oceanic scripts

American scripts

Notational systems

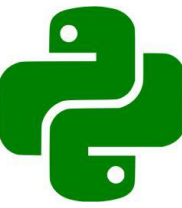
Symbols

Private use

UTF-16 surrogates

Unallocated code points

As of Unicode 13.0

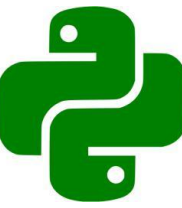


Emoji

- Emojis are visual shorthand used to convey emotion, tone, and context in digital communication.
- Unicode 17.0 specifies a total of 3,953 emoji using 1,438 characters spread across 24 blocks, of which 26 are Regional indicator symbols that combine in pairs to form flag emoji, and twelve (`<#>`, `<*>` and `<0>`–`<9>`) are base characters for keycap emoji sequences

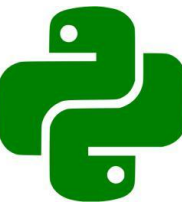
String

LECTURE 4



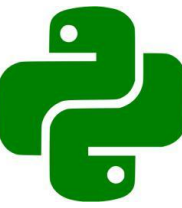
The String Data Type

- The most common use of personal computers is word processing.
- Text is represented in programs by the string data type.
- A string is a sequence of characters enclosed within quotation marks (") or apostrophes (').



The String Data Type

```
>>> str1="Hello"  
>>> str2='spam'  
>>> print(str1, str2)  
Hello spam  
>>> type(str1)  
<class 'str'>  
>>> type(str2)  
<class 'str'>
```

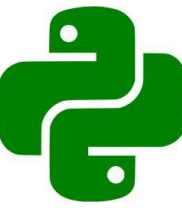


The String Data Type

- Getting a string as input

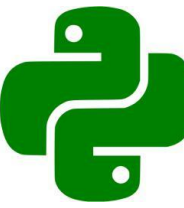
```
>>> firstName = input("Please enter your name: ")
Please enter your name: John
>>> print("Hello", firstName)
Hello John
```

- Notice that the input is not evaluated. We want to store the typed characters, not to evaluate them as a Python expression.



The String Data Type

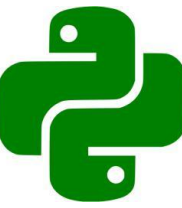
- We can access the individual characters in a string through indexing.
- The positions in a string are numbered from the left, starting with 0.
- The general form is `<string>[<expr>]`, where the value of `expr` determines which character is selected from the string.



The String Data Type

H	e	l	l	o		B	o	b
0	1	2	3	4	5	6	7	8

```
>>> greet = "Hello Bob"
>>> greet[0]
'H'
>>> print(greet[0], greet[2], greet[4])
H l o
>>> x = 8
>>> print(greet[x - 2])
B
```

The String Data Type

H	e	l	l	o		B	o	b
0	1	2	3	4	5	6	7	8

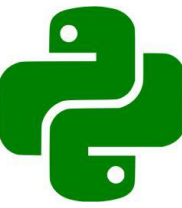
- In a string of n characters, the last character is at position $n-1$ since we start counting with 0.
- We can index from the right side using negative indexes.

```
>>> greet[-1]
```

```
'b'
```

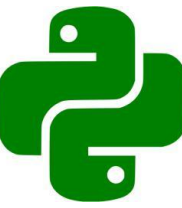
```
>>> greet[-3]
```

```
'B'
```



The String Data Type

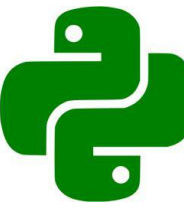
- Indexing returns a string containing a single character from a larger string.
- We can also access a contiguous sequence of characters, called a **substring**, through a process called **slicing**.



The String Data Type

Slicing:

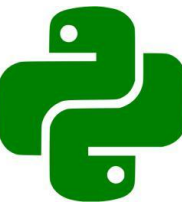
- `<string>[<start>:<end>]`
- `start` and `end` should both be ints
- The slice contains the substring beginning at position `start` and runs up to but **doesn't include** the position `end`.



The String Data Type

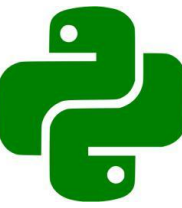
H	e	l	l	o		B	o	b
0	1	2	3	4	5	6	7	8

```
>>> greet[0:3]
'Hel'
>>> greet[5:9]
' Bob'
>>> greet[:5]
'Hello'
>>> greet[5:]
' Bob'
>>> greet[:]
'Hello Bob'
```



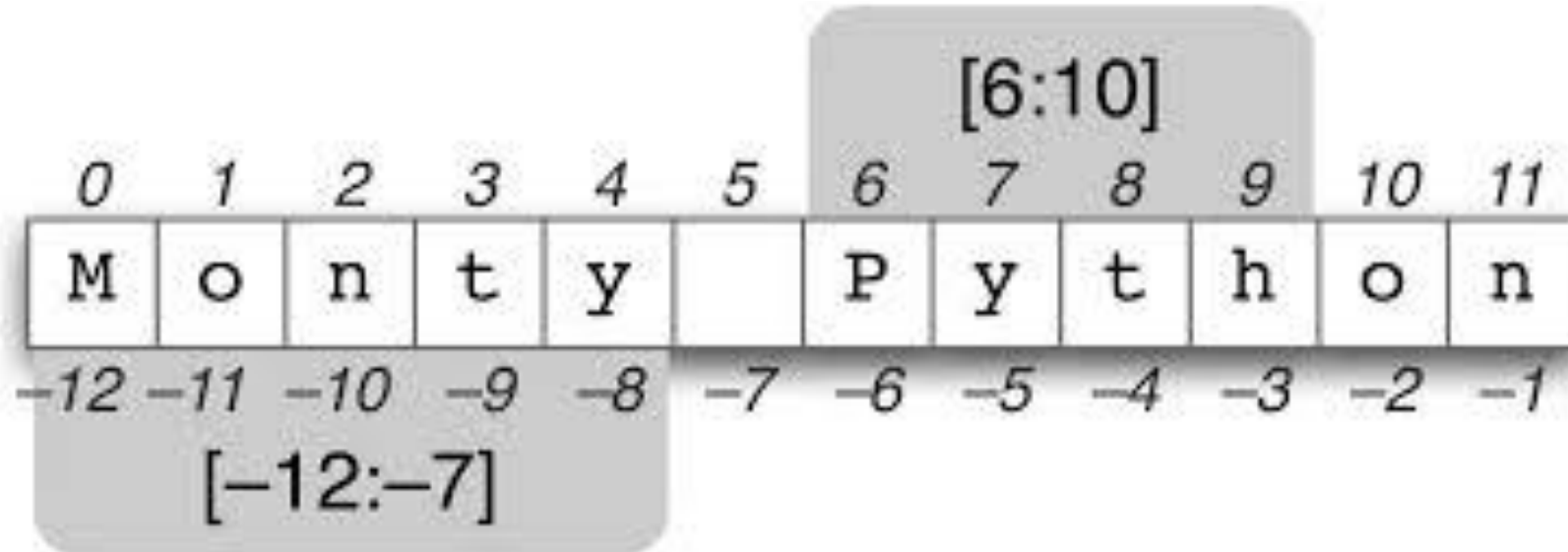
The String Data Type

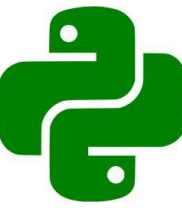
- If either expression is missing, then the start or the end of the string are used.
- Can we put two strings together into a longer string?
- Concatenation “glues” two strings together (+)
- Repetition builds up a string by multiple concatenations of a string with itself (*)



The String Data Type

- Text in a program is represented by the string data type.
- String can also be viewed as an linear (1-D) array of characters.
- String can also be viewed as a sequence of characters.

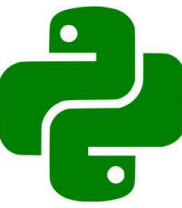




The String Data Type

- The function `len` will return the length of a string.

```
>>> "spam" + "eggs"
'spameggs'
>>> "Spam" + "And" + "Eggs"
'SpamAndEggs'
>>> 3 * "spam"
'spamspamspam'
>>> "spam" * 5
'spamspamspamspamspam'
>>> (3 * "spam") + ("eggs" * 5)
'spamspamspaceggseggsseggsseggs'
```



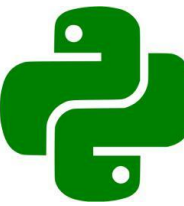
The String Data Type

```
>>> len("spam")
```

```
4
```

```
>>> for ch in "Spam!":  
        print (ch, end=" ")
```

```
S p a m !
```

The String Data Type

Operator	Meaning
+	Concatenation
*	Repetition
<string>[]	Indexing
<string>[:]	Slicing
len(<string>)	Length
for <var> in <string>	Iteration through characters

One-Way Selection

LECTURE 5

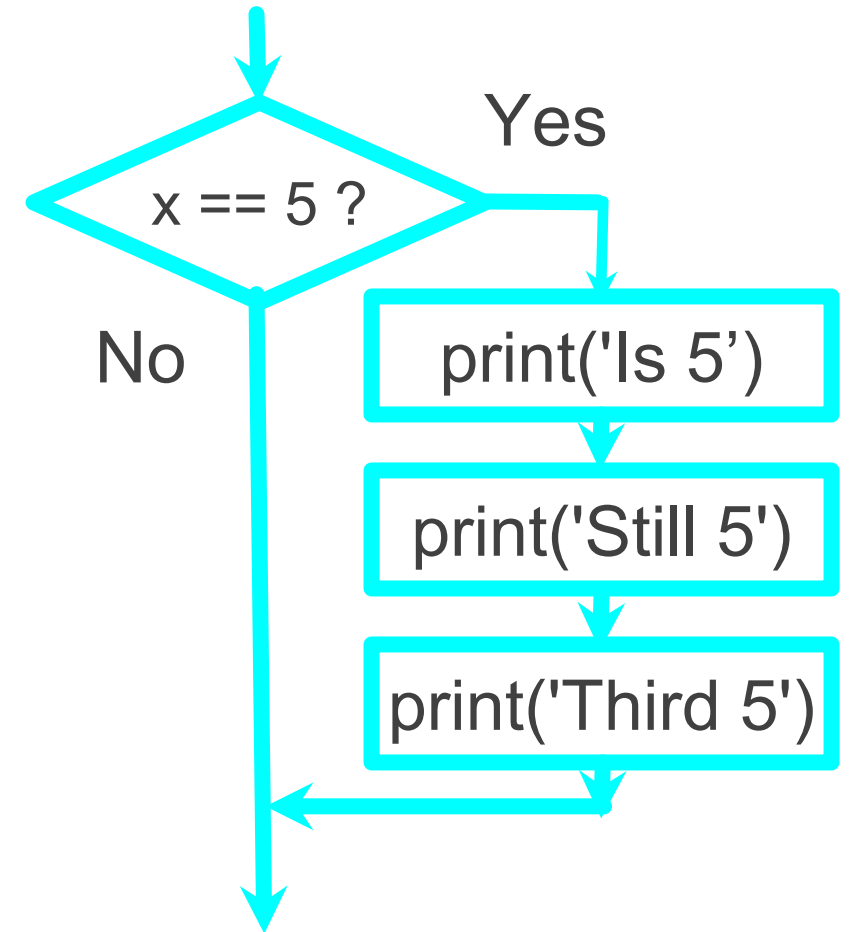
One-Way Decisions

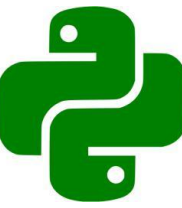
```
x = 5
print('Before 5')
if x == 5 :
    print('Is 5')
    print('Is Still 5')
    print('Third 5')
print('Afterwards 5')
print('Before 6')
if x == 6 :
    print('Is 6')
    print('Is Still 6')
    print('Third 6')
print('Afterwards 6')
```

Before 5

Is 5
Is Still 5
Third 5
Afterwards 5
Before 6

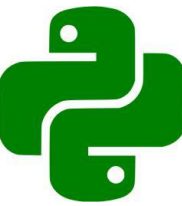
Afterwards 6





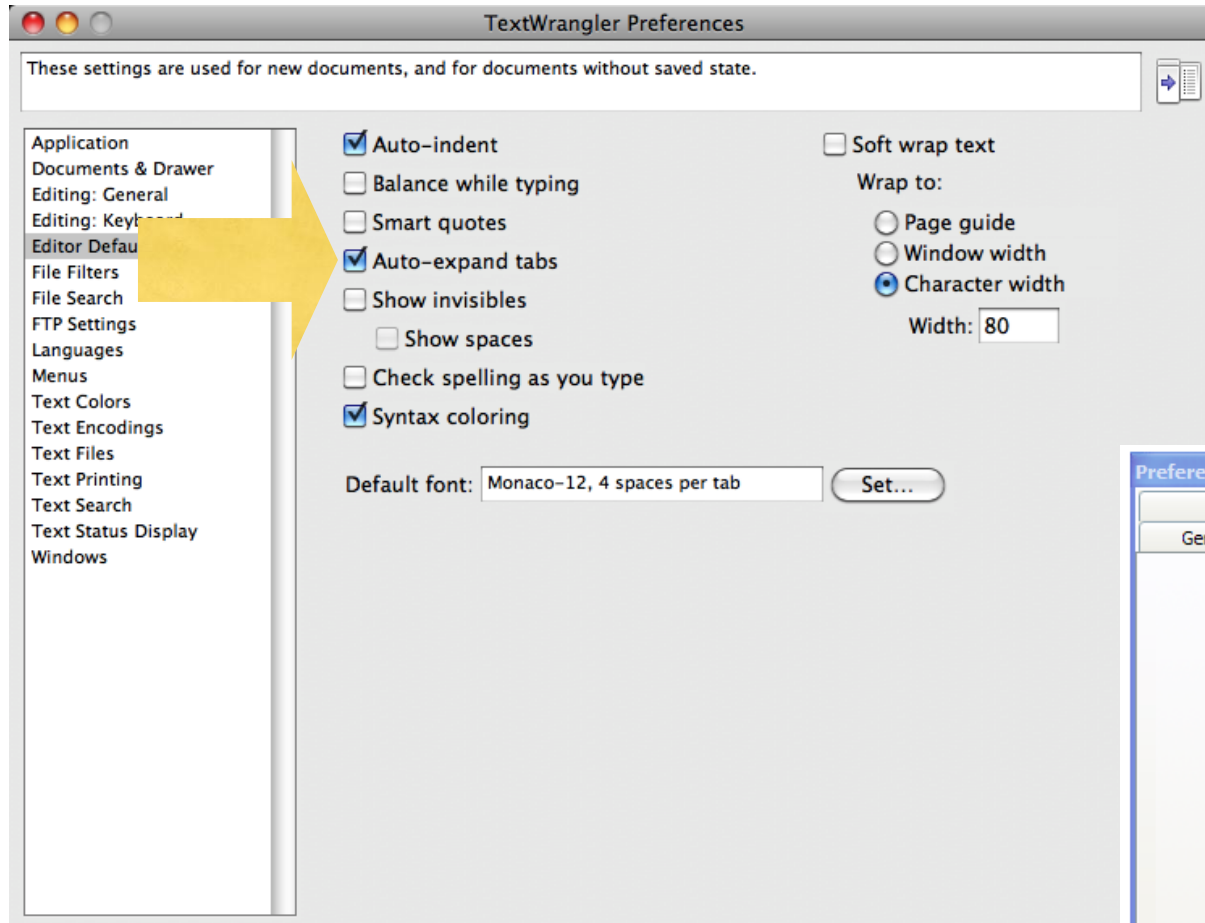
Indentation

- Increase indent after an if statement or for statement (after :)
- Maintain indent to indicate the scope of the block (which lines are affected by the if/for)
- Reduce indent back to the level of the if statement or for statement to indicate the end of the block
- Blank lines are ignored - they do not affect indentation
- Comments on a line by themselves are ignored with regard to indentation

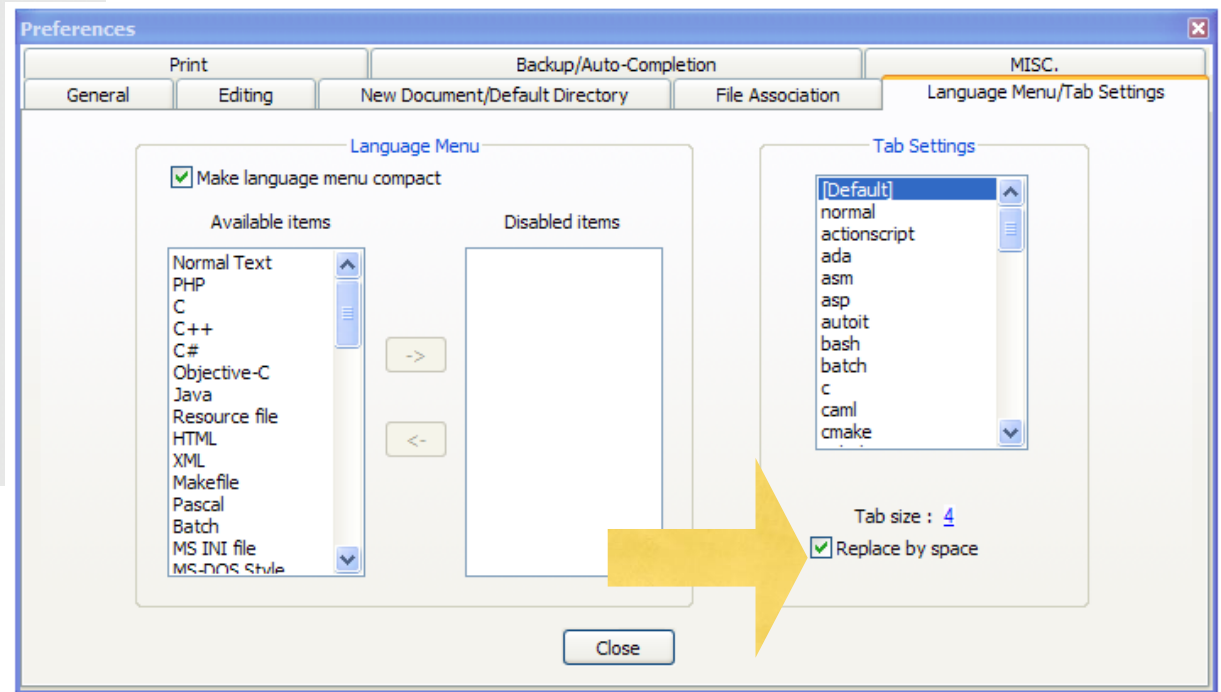


Warning: Turn Off Tabs!!

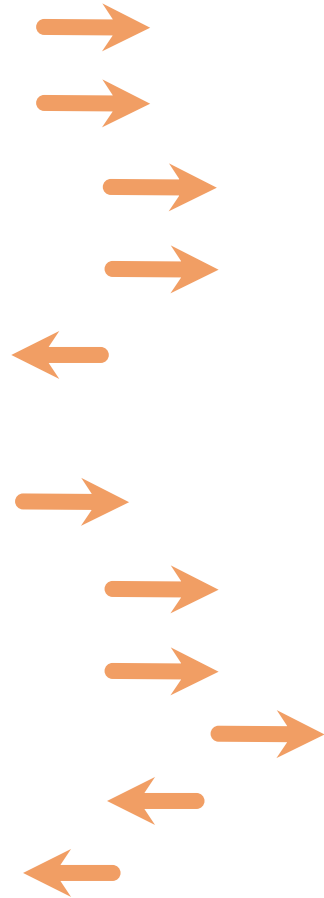
- Atom automatically uses spaces for files with ".py" extension (nice!)
- Most text editors can turn tabs into spaces - make sure to enable this feature
 - - Notepad++: Settings -> Preferences -> Language Menu/Tab Settings
 - - TextWrangler: TextWrangler -> Preferences -> Editor Defaults
- Python cares a *lot* about how far a line is indented. If you mix tabs and spaces, you may get "indentation errors" even if everything looks fine



This will save you
much unnecessary
pain.



increase / maintain after if or for
decrease to indicate end of block



```
x = 5
if x > 2 :
    print('Bigger than 2')
    print('Still bigger')
print('Done with 2')

for i in range(5) :
    print(i)
    if i > 2 :
        print('Bigger than 2')
    print('Done with i', i)
print('All Done')
```

Think About begin/end Blocks

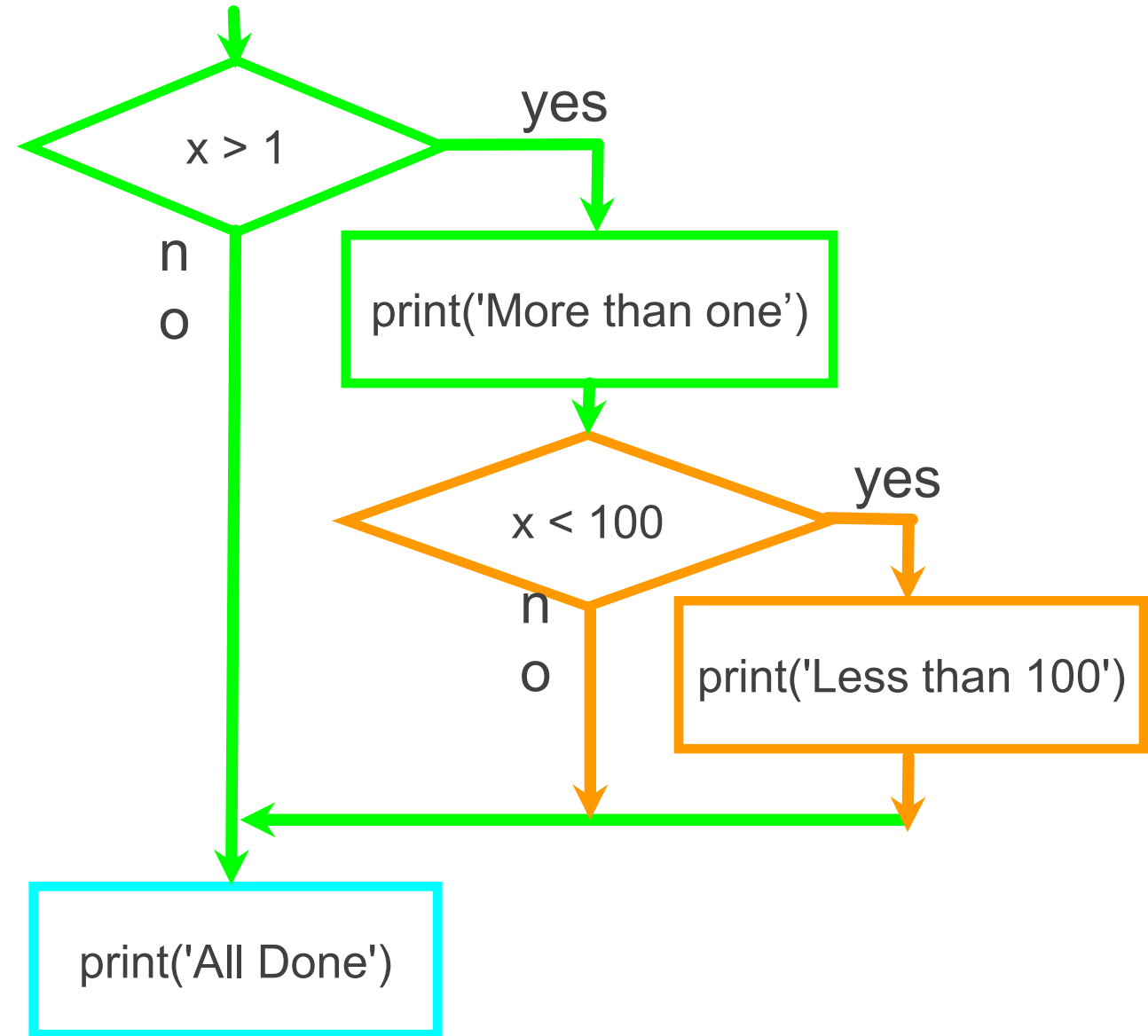
```
x = 5
```

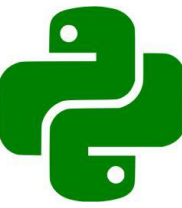
```
if x > 2 :  
    print('Bigger than 2')  
    print('Still bigger')  
print('Done with 2')
```

```
for i in range(5) :  
    print(i)  
    if i > 2 :  
        print('Bigger than 2')  
    print('Done with i', i)  
print('All Done')
```


Nested Decisions

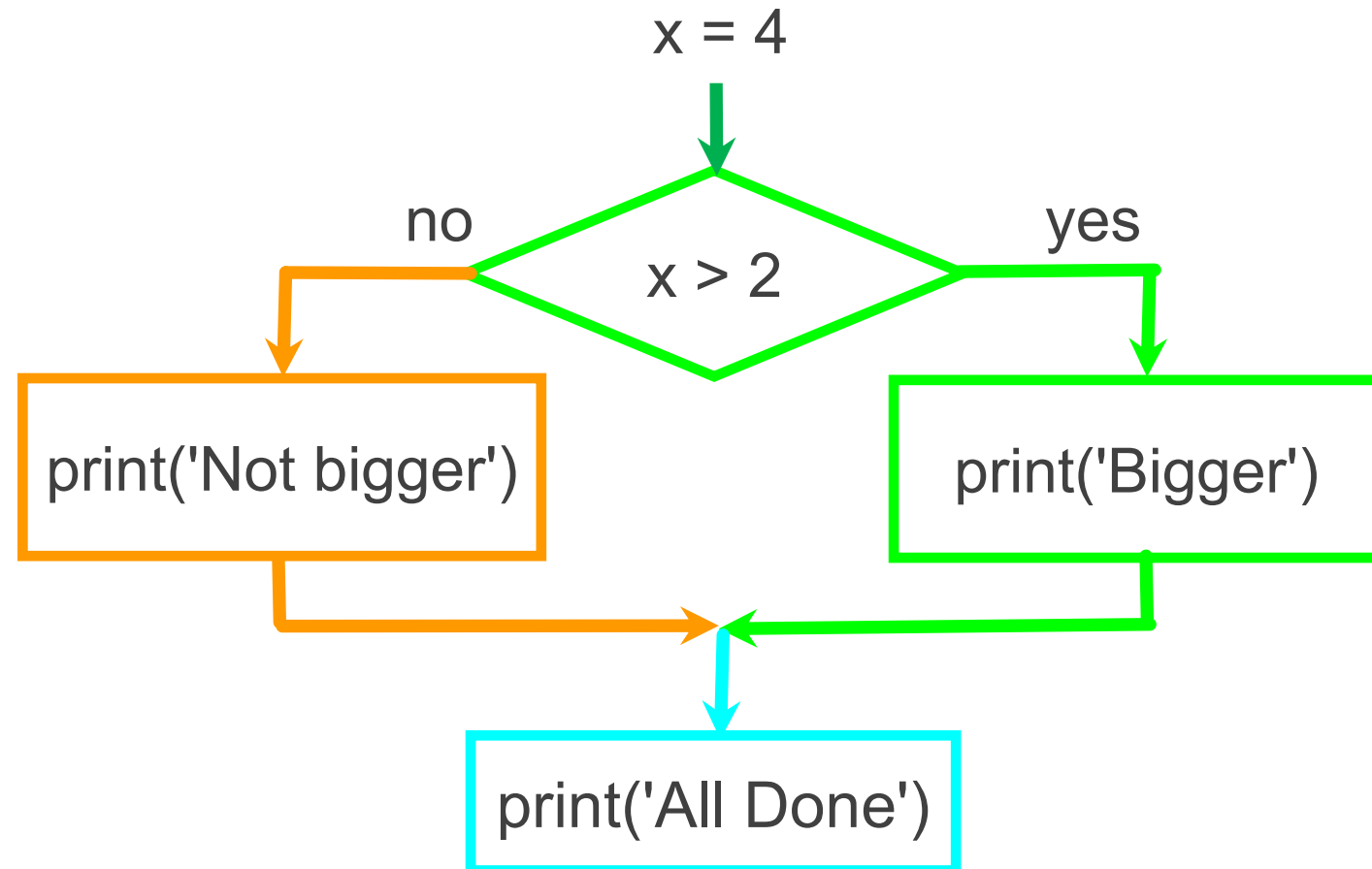
```
x = 42
if x > 1 :
    print('More than one')
    if x < 100 :
        print('Less than 100')
print('All done')
```

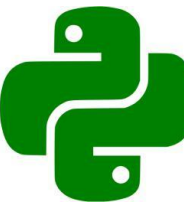




Two-way Decisions

- Sometimes we want to do one thing if a logical expression is true and something else if the expression is false
- It is like a fork in the road - we must choose one or the other path but not both

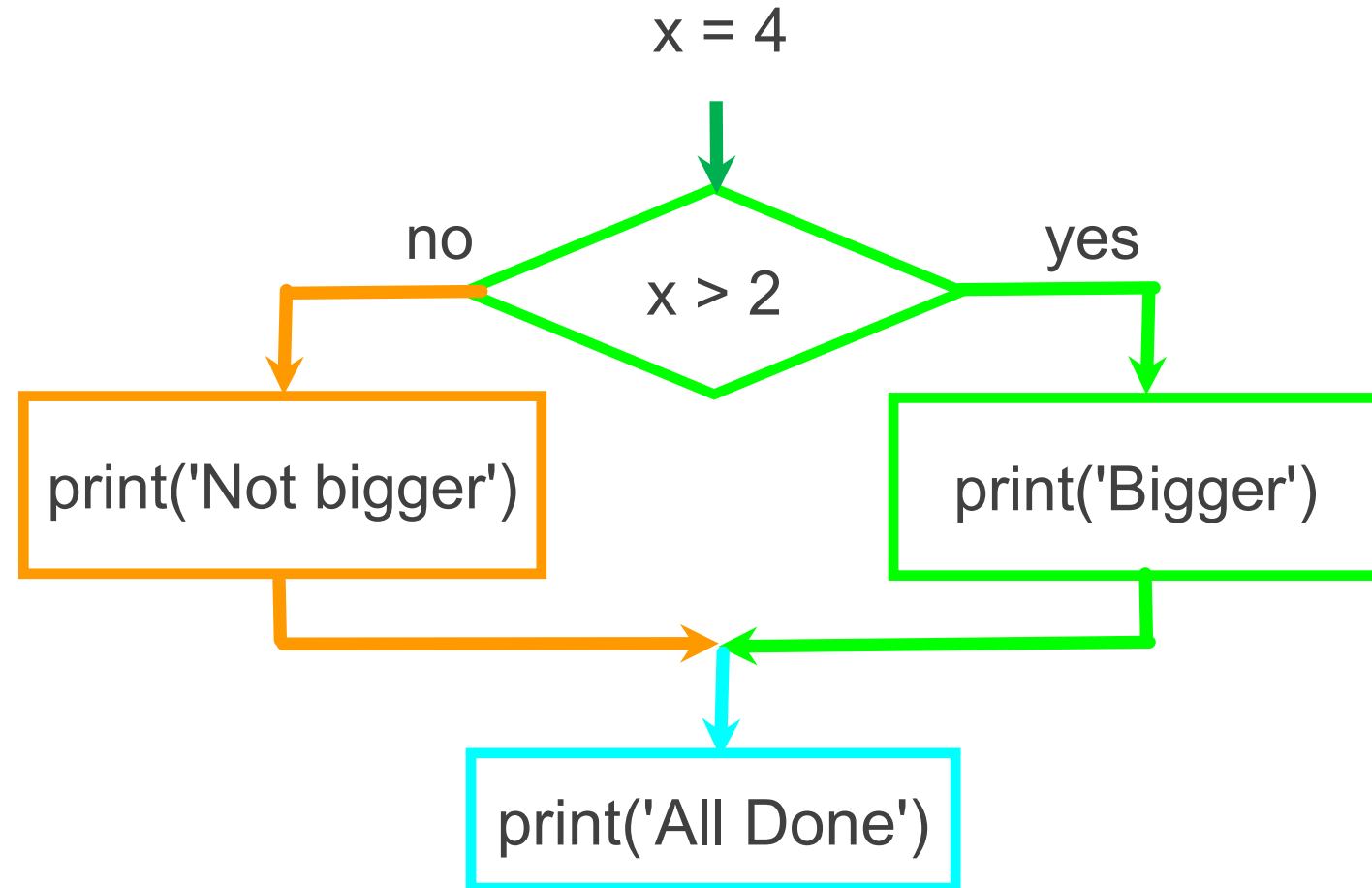


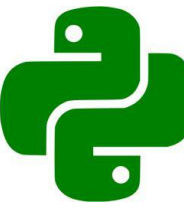


Two-way Decisions with else:

```
x = 4
```

```
if x > 2 :  
    print('Bigger')  
else :  
    print('Smaller')  
  
print('All done')
```



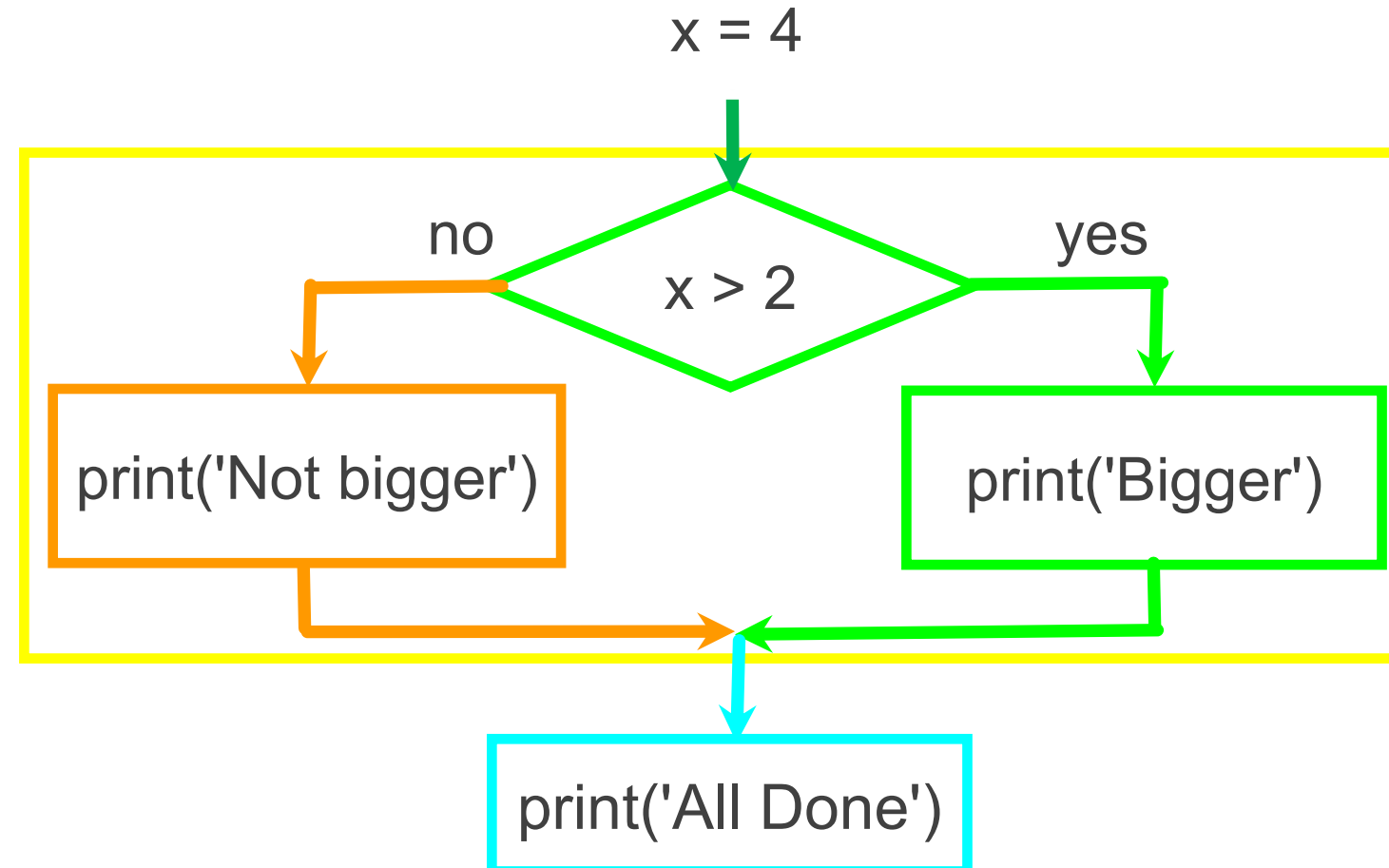


Visualize Blocks

```
x = 4
```

```
if x > 2 :  
    print('Bigger')  
else :  
    print('Smaller')
```

```
print('All done')
```

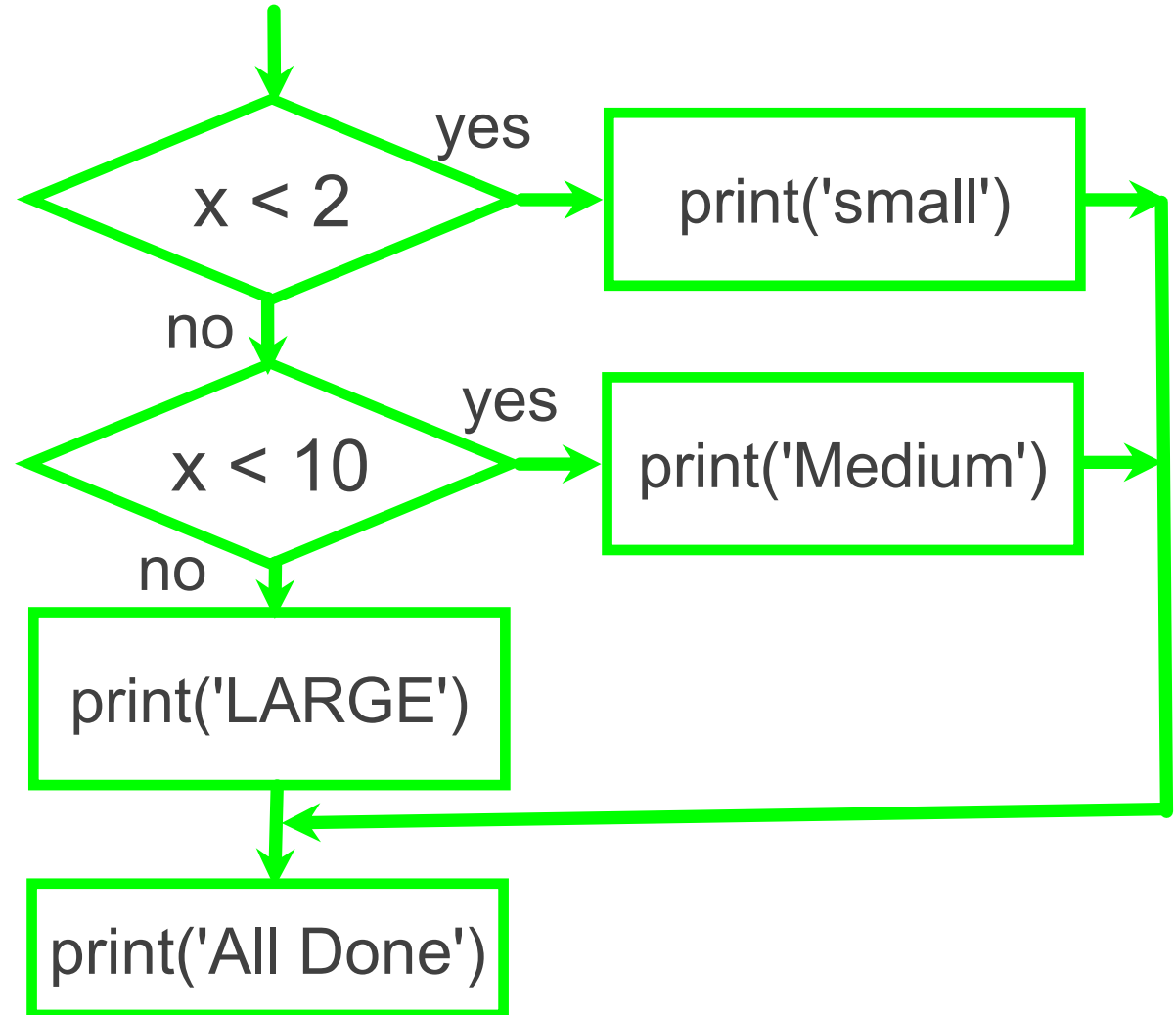


Multiple-Way Selection

LECTURE 6

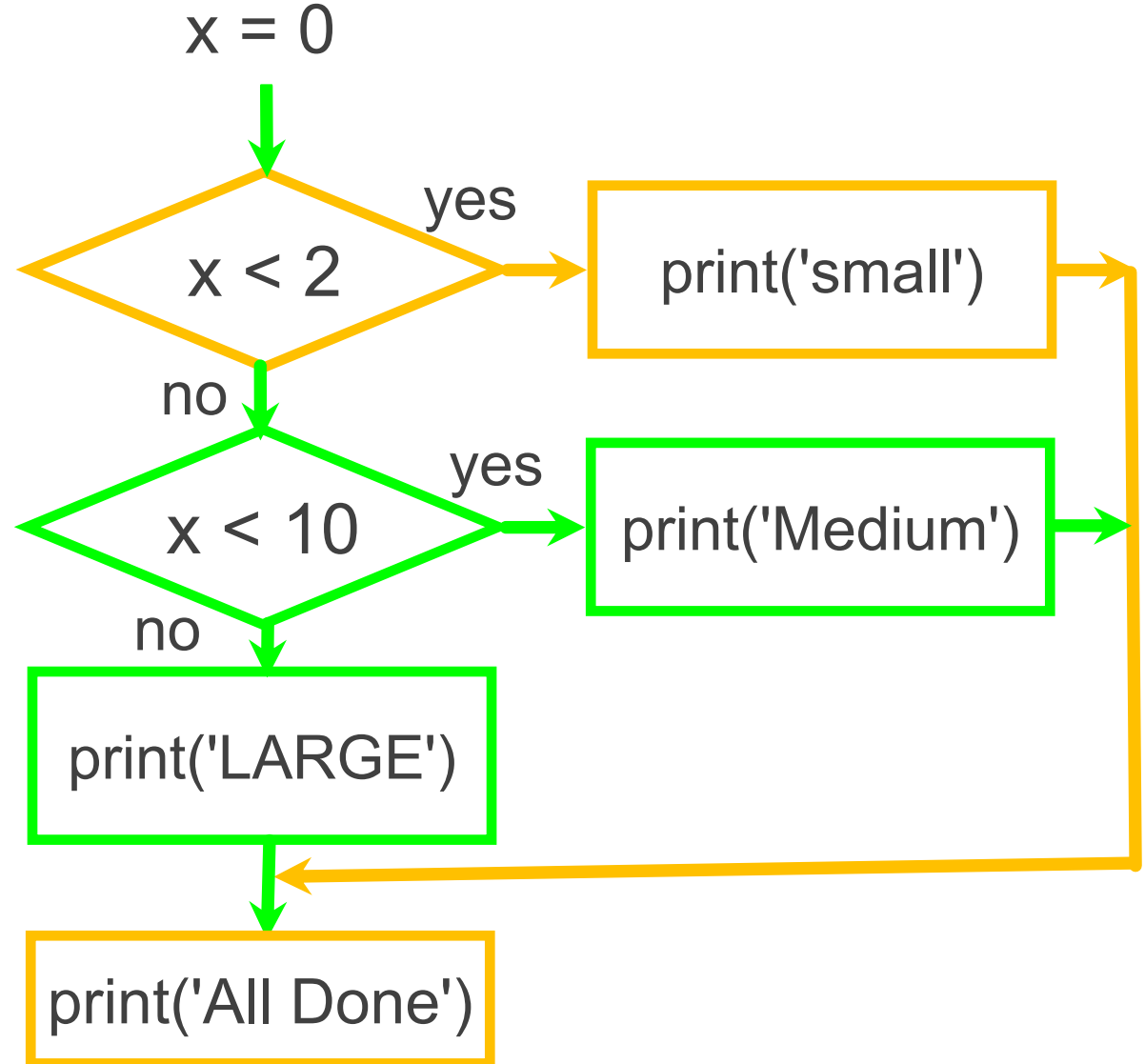
Multi-way

```
if x < 2 :  
    print('small')  
elif x < 10 :  
    print('Medium')  
else :  
    print('LARGE')  
print('All done')
```



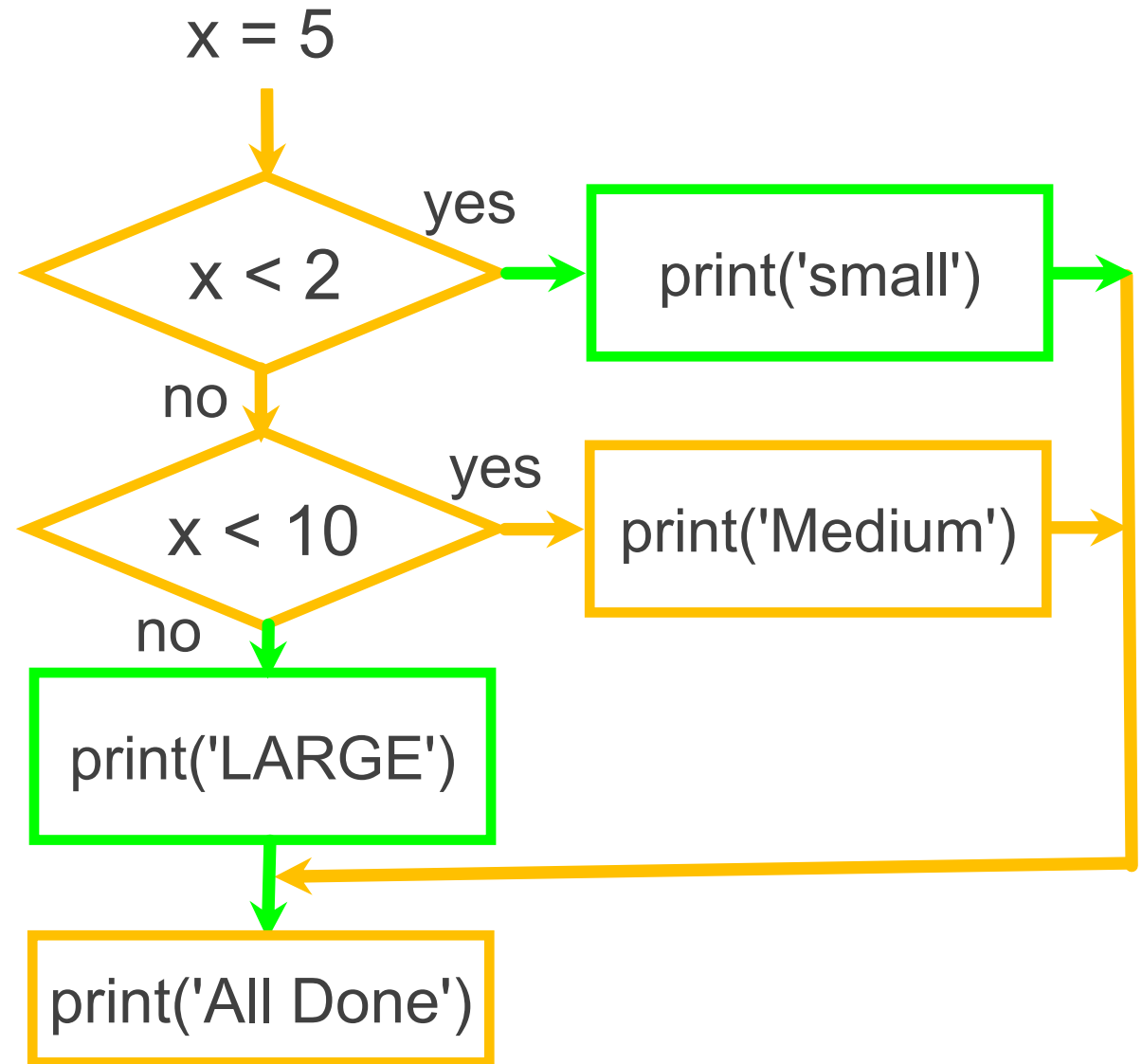
Multi-way

```
x = 0
if x < 2 :
    print('small')
elif x < 10 :
    print('Medium')
else :
    print('LARGE')
print('All done')
```



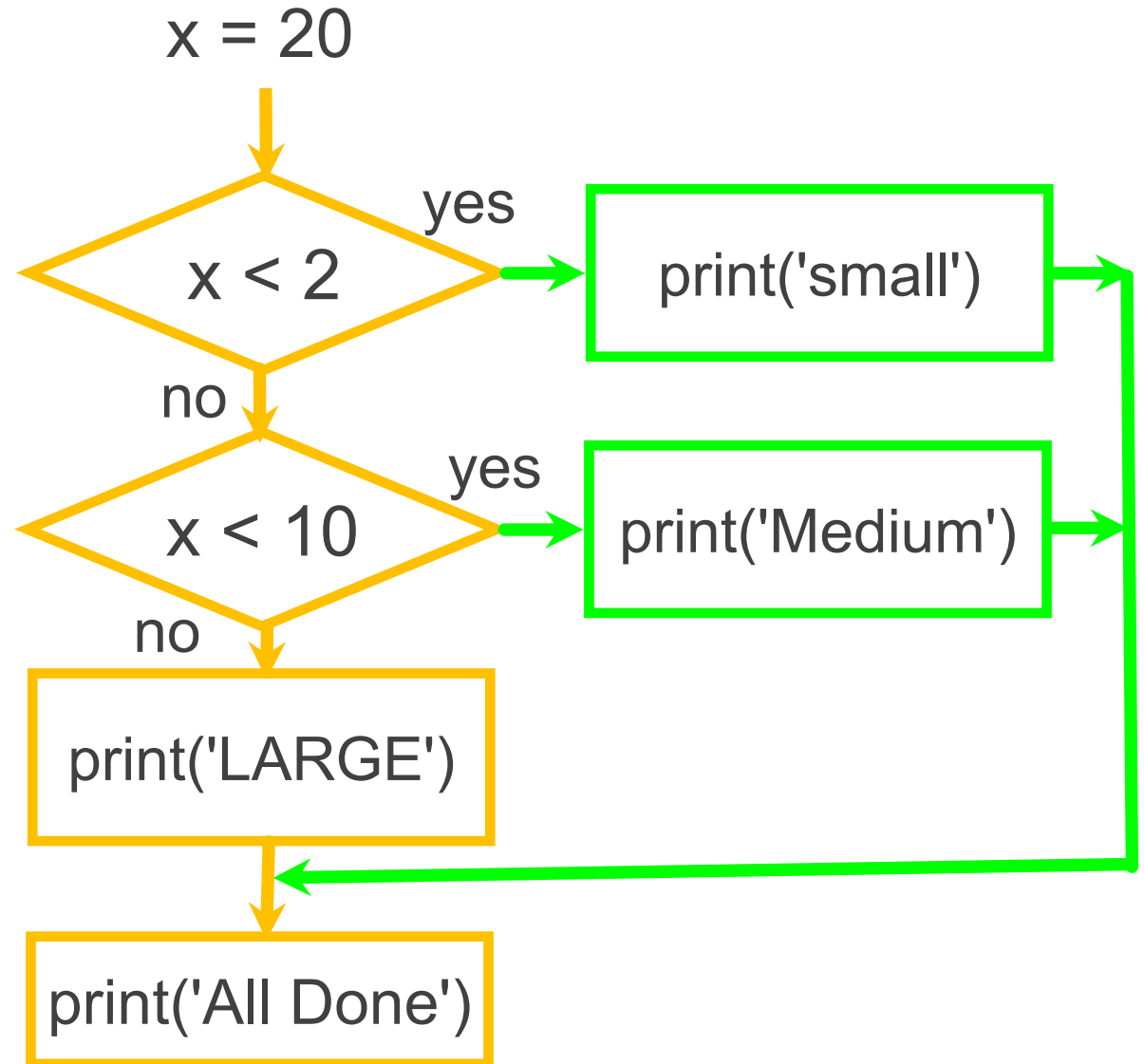
Multi-way

```
x = 5
if x < 2 :
    print('small')
elif x < 10 :
    print('Medium')
else :
    print('LARGE')
print('All done')
```



Multi-way

```
x = 20
if x < 2 :
    print('small')
elif x < 10 :
    print('Medium')
else :
    print('LARGE')
print('All done')
```

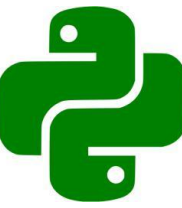


Multi-way

```
# No Else
x = 5
if x < 2 :
    print('Small')
elif x < 10 :
    print('Medium')

print('All done')
```

```
if x < 2 :
    print('Small')
elif x < 10 :
    print('Medium')
elif x < 20 :
    print('Big')
elif x < 40 :
    print('Large')
elif x < 100:
    print('Huge')
else :
    print('Ginormous')
```



Multi-way Puzzles

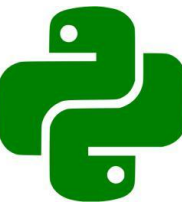
Which will never print regardless of the value for x?

```
if x < 2 :  
    print('Below 2')  
elif x >= 2 :  
    print('Two or more')  
else :  
    print('Something else')
```

```
if x < 2 :  
    print('Below 2')  
elif x < 20 :  
    print('Below 20')  
elif x < 10 :  
    print('Below 10')  
else :  
    print('Something else')
```

Conditional Statement

LECTURE 7



Conditional Statement

value_for_true if boolean_expression **else** **value_for_false**

a = 3 if x > 4 else 4

○

a will get a value of 3 if x > 4.

otherwise, it will get 4.



Lab

SALARY

Exercise

Rewrite your pay computation to give the employee 1.5 times the hourly rate for hours worked above 40 hours.

Enter Hours: 45

Enter Rate: 10

Pay: 475.0

$$475 = 40 * 10 + 5 * 15$$

Exercise

Rewrite your pay program using try and except so that your program handles non-numeric input gracefully.

```
Enter Hours: 20
```

```
Enter Rate: nine
```

```
Error, please enter numeric input
```

```
Enter Hours: forty
```

```
Error, please enter numeric input
```